

Roisy's Blog

一个SE学生的自留地

🏠 首页

📁 归档

🏷 标签

👤 关于

BUAA-OS-Lab4 系统调用与fork

📅 2024-05-08 | 📁 BUAA-SE | 📖 41 阅读 | 📄 2.5k 字 | ⌚ 9 分钟

文章目录

1. 指导书梳理

1.1. 系统调用

1.1.1. 系统调用原理-步骤

1.1.2. 系统调用机制的实现

1.1.3. 基础系统调用函数

1.2. 进程间通信机制 (IPC)

1.3. Fork

1.3.1. fork基础

1.3.2. 写时复制

1.3.3. 页写入异常

2. 时纪

3. 上机准备

3.0.1. 需时刻注意区分用户态和内核态

3.0.2. 常用函数/用法

3.0.3. 往年教训

4. 上机血泪教训

1. 4.1. extra-1

2. 4.2. exam-2

关键文件: kern/syscall_all.c、user/lib/fork.c、include/syscall.h、user/include/lib.h

指导书梳理

系统调用



系统调用原理-步骤

1. 存在一些只能由内核来完成的操作（如读写设备、创建进程、IO 等）。
2. C 标准库中一些函数的实现须依赖于操作系统（如我们所探究的 puts 函数）。
3. 通过执行 syscall 指令，用户进程可以陷入到内核态，请求内核提供的服务。
4. 通过系统调用陷入到内核态时，需要在用户态与内核态之间进行数据传递与保护。

系统调用机制的实现

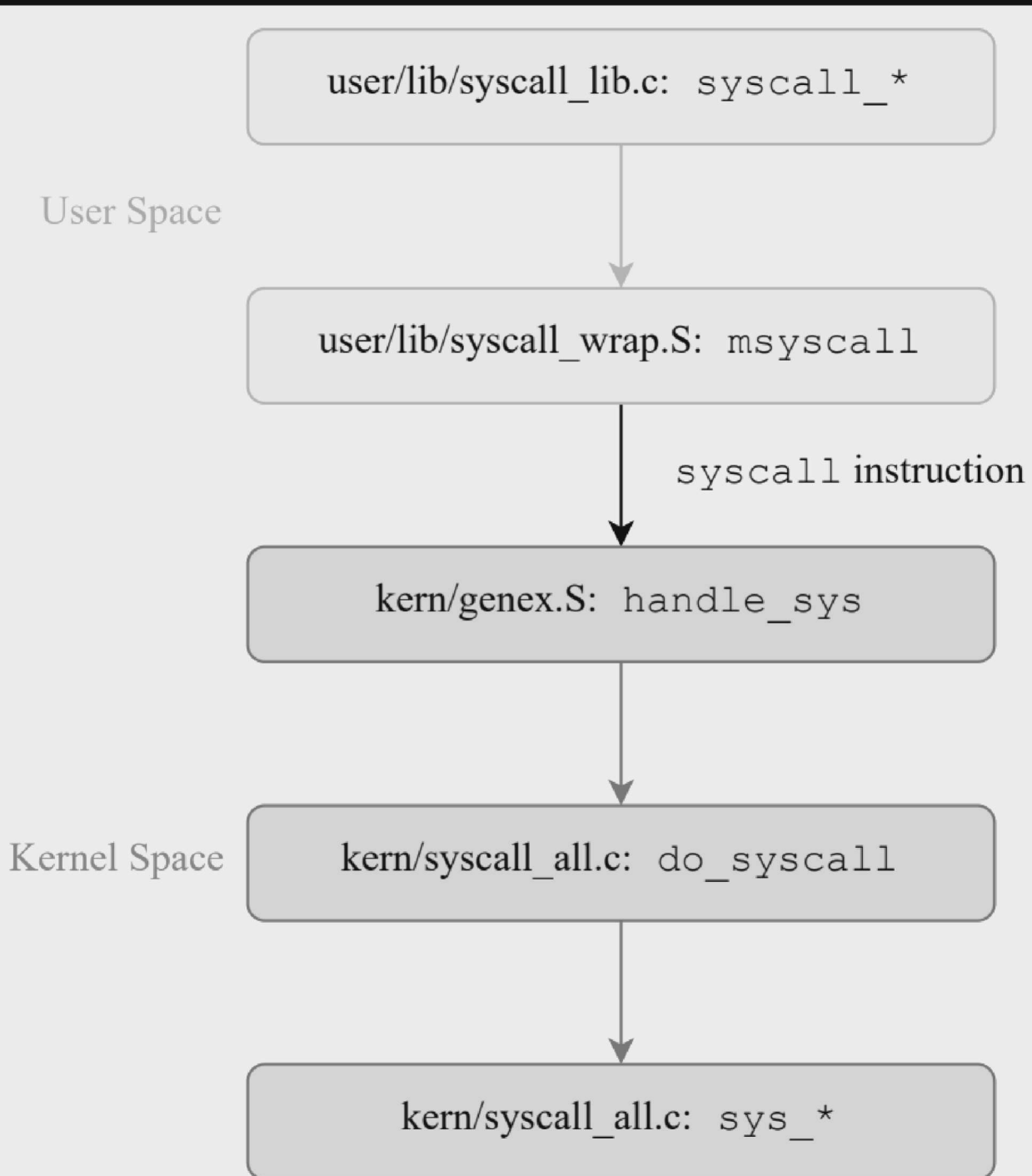


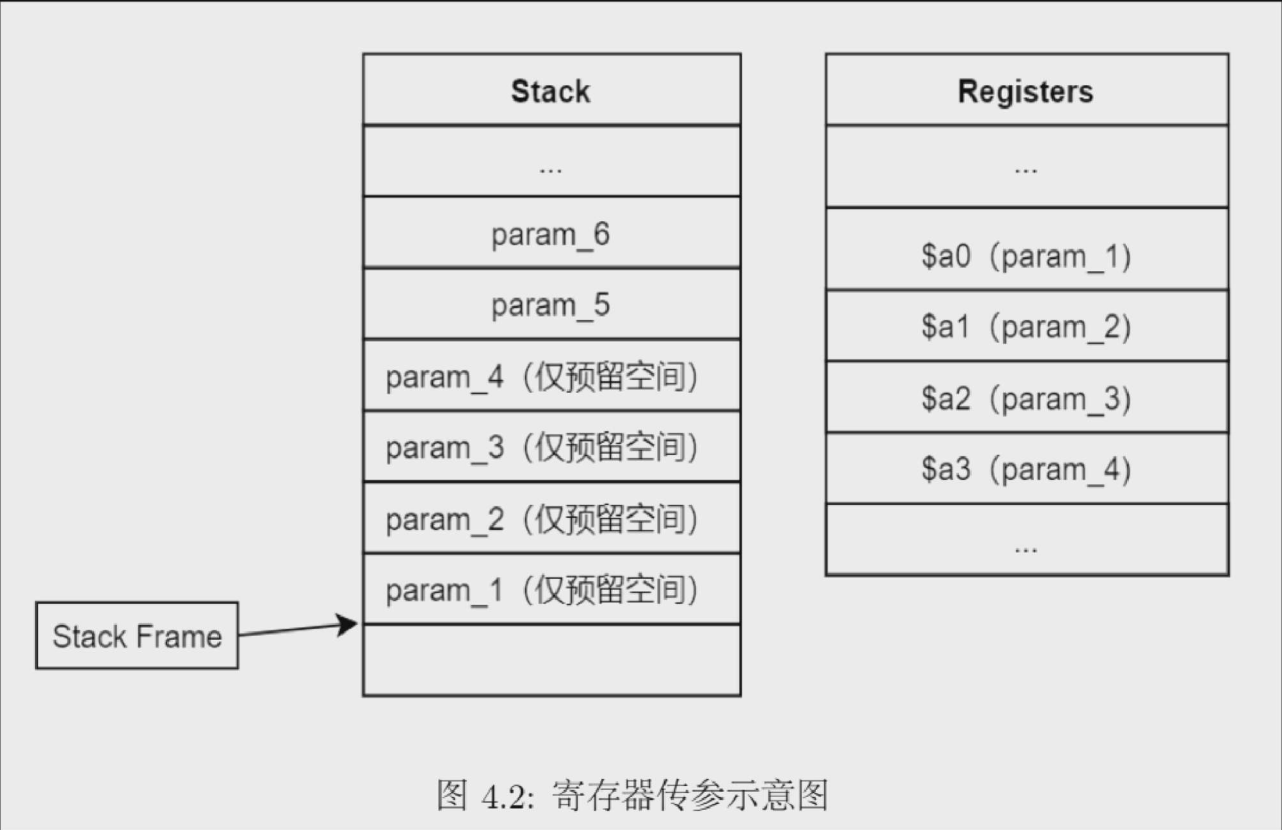
图 4.1: syscall 过程流程图

• **syscall_***

- 在 MOS 中， syscall_* 的函数与内核中的系统调用函数（sys_* 的函数）是一一对应的： syscall_* 的函数是用户空间中最接近的内核的函数，而 sys_* 的函数是内核中系统调用的具体实现部分。
- syscall_* 的函数的实现中，毫无例外都调用了 msyscall 函数，而且函数的第一个参数都是一个与调用名相似的宏（如 SYS_print_cons），MOS 中把这个参数称为**系统调用号**，它们被定义在 include/syscall.h 中

• **msyscall 函数——参数传递的实现**

- 进入函数体时会通过对栈指针做减法（压栈）的方式为该函数自身的局部变量、返回地址、调用函数的参数分配存储空间，在函数调用结束之后会对栈指针做加法（弹栈）释放这部分空间，这部分空间称为**栈帧**。调用方在自身栈帧的底部预留被调用函数的参数存储空间，由被调用方从调用方的栈帧中读取参数。
- 寄存器 a0—a3 用于存放函数调用的前四个参数
- 剩余的参数仅存放在栈中，但在栈中仍然需要为存放在寄存器中的参数预留空间



Untitled

- 系统从用户态切换到内核态后，内核首先将原用户程序的运行现场保存到内核空间（在 kern/entry.S 中通过 SAVE_ALL 宏完成），随后**内核空间的栈指针则指向保存的 Trapframe**，便可以借助这个保存的结构体来获取用户态中传递过来的值

基础系统调用函数

在内核处理进程发起的系统调用时，我们并没有切换地址空间（页目录地址），也不需要将进程上下文（Trapframe）保存到进程控制块中，只是切换到内核态下，执行了一些内核代码。

可以说，处理系统调用时的内核仍然是代表当前进程的，这也是系统调用、TLB 缺失等同步异常与时钟中断等异步异常的本质区别

进程间通信机制 (IPC)

- IPC的关键
 - IPC 的目的是使两个进程之间可以通信

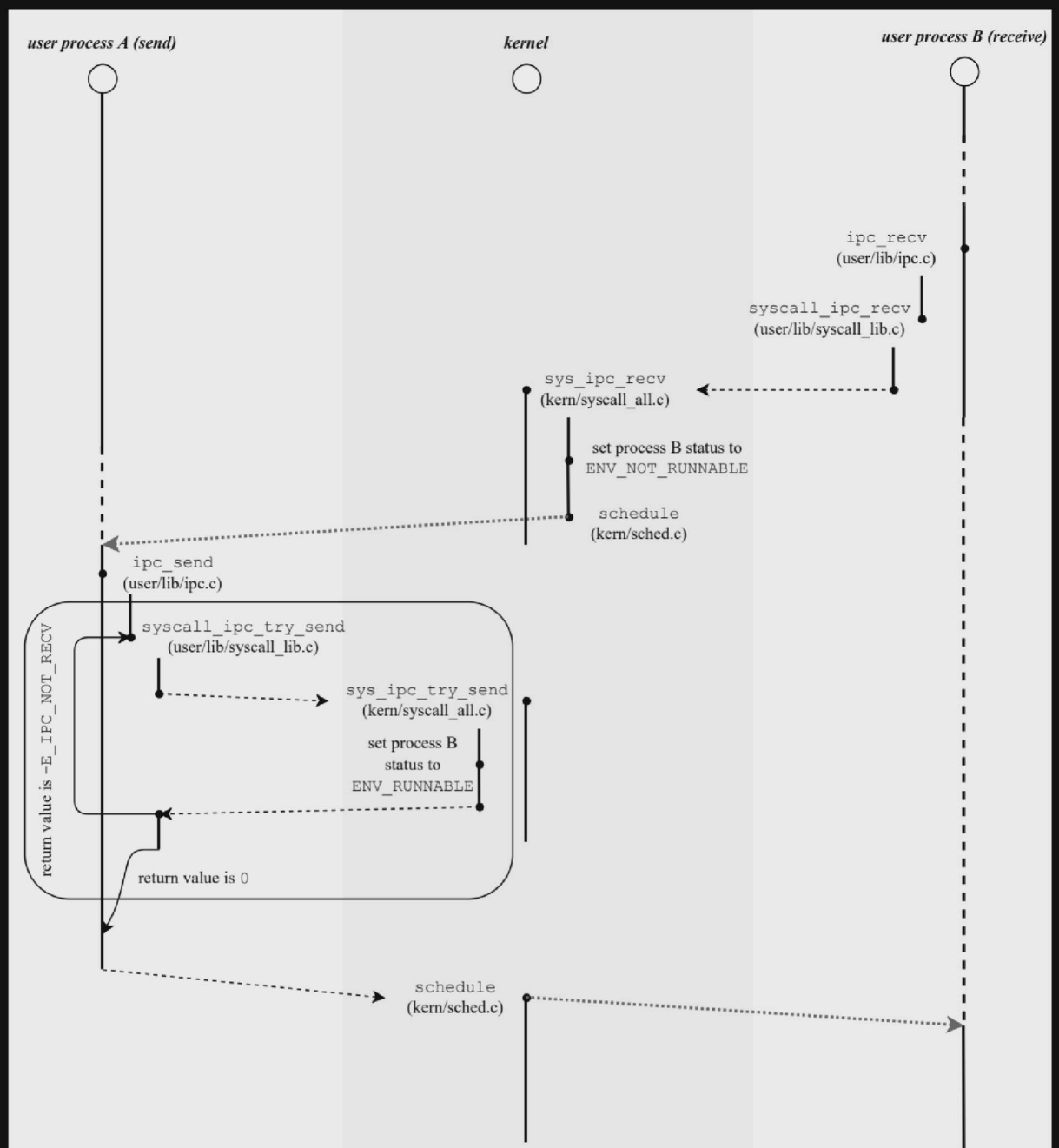
通俗的讲，就是把一个地址空间中的东西传给另一个地址空间

- IPC 需要通过系统调用来实现

所有的进程都共享同一个内核空间（主要为 kseg0）。因此，想要在不同空间之间交换数据，就可以借助于内核空间来实现。

发送方进程可以将数据以系统调用的形式存放在进程控制块中，接收方进程同样以系统调用的方式在进程控制块中找到对应的数据，读取并返回。

env_ipc_value	进程传递的具体数值
env_ipc_from	发送方的进程 ID
env_ipc_recving	1：等待接受数据中；0：不可接受数据
env_ipc_dstva	接收到的页面需要与自身的哪个虚拟页面完成映射
env_ipc_perm	传递的页面的权限位设置



Untitled

Fork

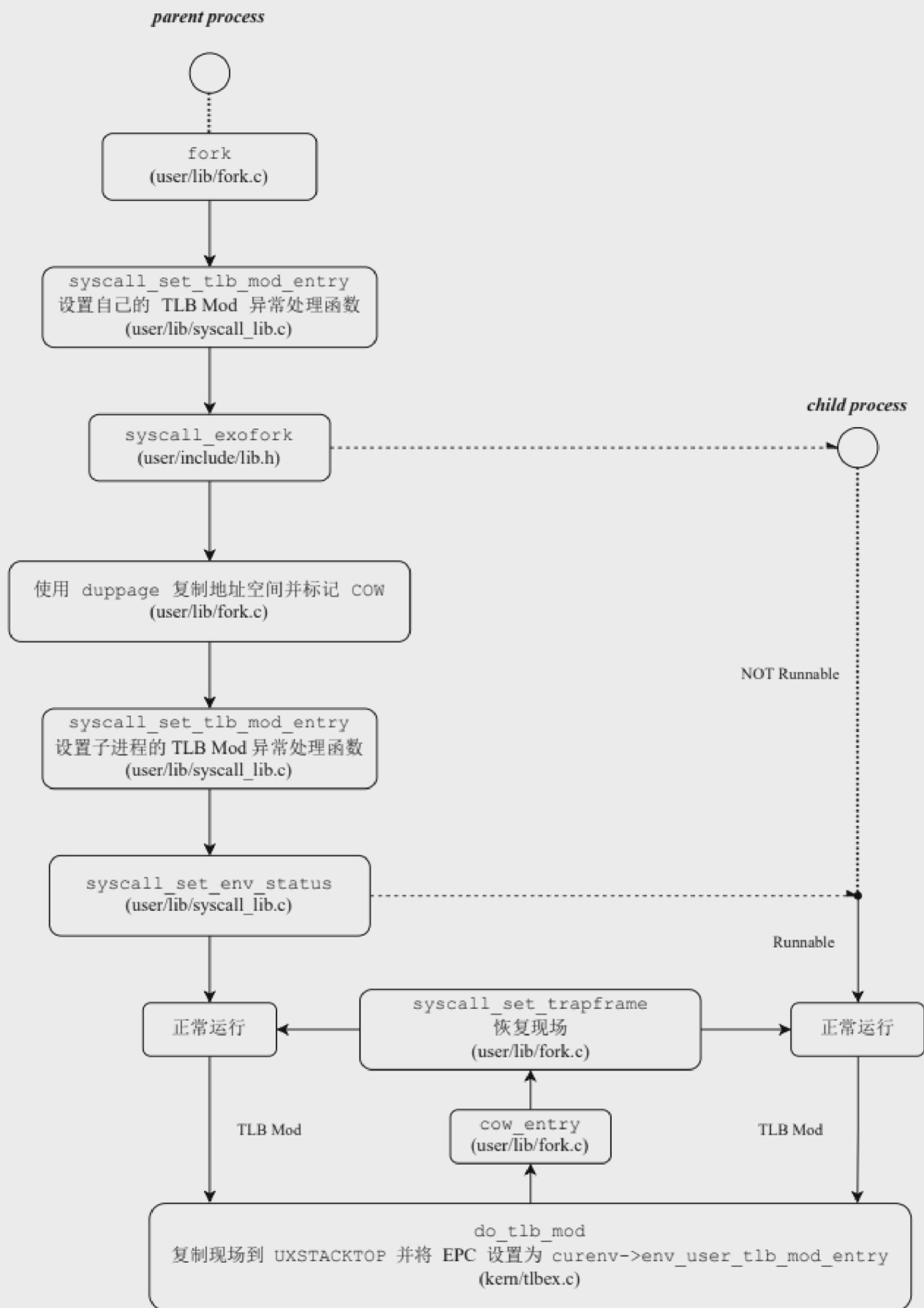


图 4.4: fork 流程图

Untitled

- **exec**

与 fork 经常一起使用的是名为 exec 的一系列系统调用。它会使进程抛弃现有的程序和运行现场，**执行一个新的程序**

写时复制

在 fork 时，我们只需将地址空间中的**所有可写页标记为写时复制页面**，使得在父进程或子进程对写时复制页面进行写入时，能够产生一种异常。操作系统在异常处理时，为当前进程**试图写入的**虚拟地址分配新的物理页面，并复制原页面的内容，最后再返回用户程序，对新分配的物理页面进行写入。

- 进程调用 fork 时，需要对其所有的可写入的内存页面，设置页表项标志位 PTE_COW 并取消可写位 PTE_D
 - 将写时复制界面的 PTE_D 标志置为 0

当进程读这个页面时，不会出现问题。但当进程尝试写这个页面时，由于 PTE_D 为 0，所以会触发 TLB Mod 异常，陷入内核执行写时复制流程

- 引入新的标志位 PTE_COW，为 1 则需要进行上述的写时复制处理

利用 TLB 项中的软件保留位，区分真正的“只读”页面和“写时复制”页面

页写入异常

- do_tlb_mod 函数并没有进行页面复制等 COW 的处理操作。事实上，我们的 MOS 操作系统按照微内核的设计理念，**尽可能地将功能实现在用户空间中**，其中也包括了页写入异常的处理，因此主要的处理过程是在用户态下完成的
- 如果需要在用户态下完成页写入异常的处理，是不能直接使用正常情况下的用户栈的（因为发生页写入异常的也可能是正常栈的页面），所以用户进程就需要一个单独的栈来执行处理程序，我们把这个栈称作 **异常处理栈**，它的栈顶对应的是内存布局中的 **UXSTACKTOP**
- 处理页写入异常的大致流程

1. 用户进程触发页写入异常，陷入到**内核中的 handle_mod**，再跳转到 do_tlb_mod 函数



2. `do_tlb_mod` 函数负责**将当前现场保存在异常处理栈中**，并设置 `a0` 和 `EPC` 寄存器的值，使得从异常恢复后能够以异常处理栈中保存的现场为参数，跳转到 `env_user_tlb_mod_entry` 域存储的用户异常处理函数的地址
3. 从异常恢复到用户态，跳转到用户异常处理函数 `cow_entry` (`fork.c` 中定义) 中，由用户程序完成写时复制等自定义处理。这个函数进行写时复制处理之后，使用系统调用 `syscall_set_trapframe` 恢复事先保存好的现场，其中也包括 `sp` 和 `PC` 寄存器的值，使得用户程序恢复执行

内核只是存相关寄存器做准备，用户进程从相关寄存器中取得需要数据进行实际处理

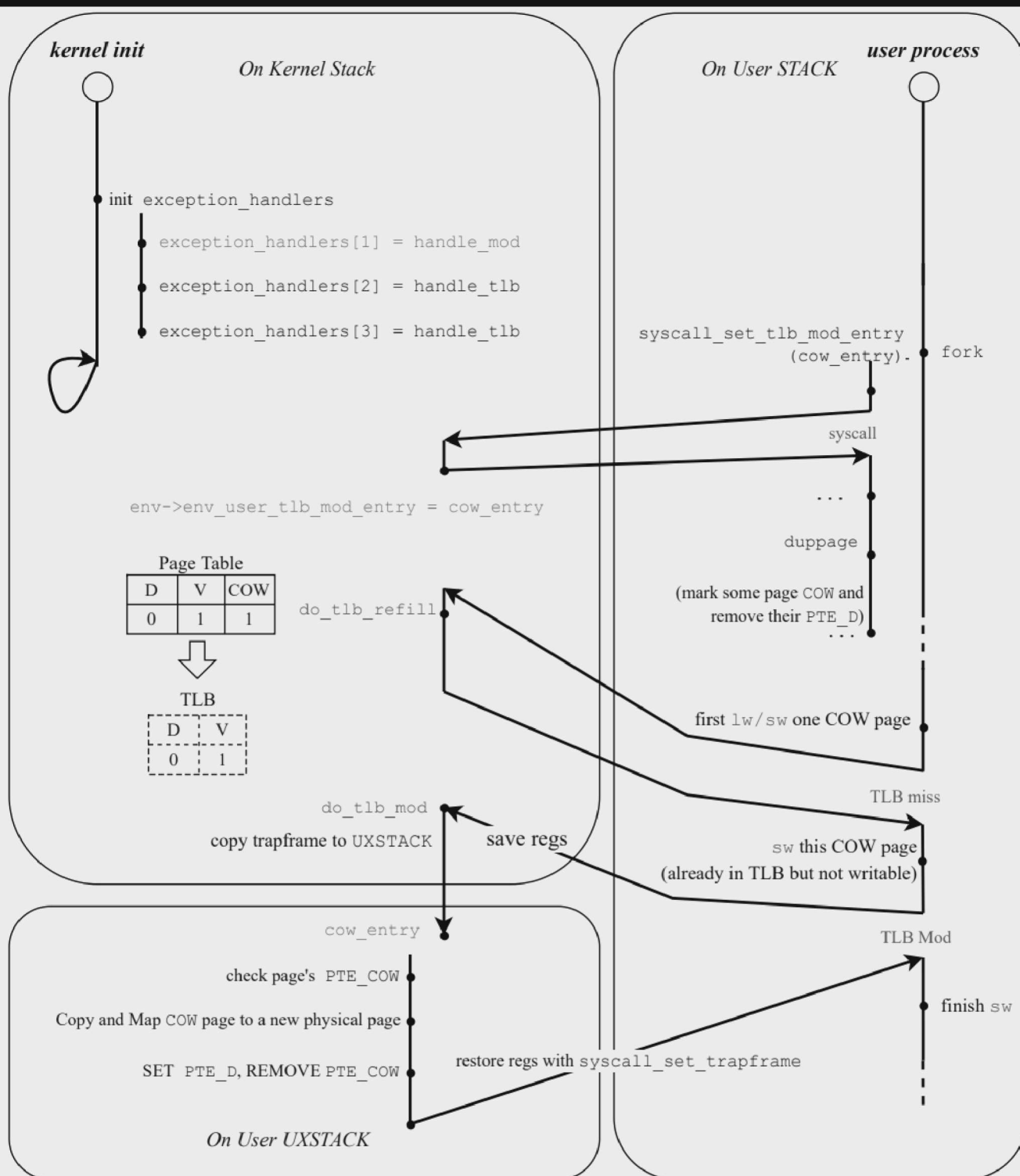


图 4.5: 页写入异常处理流程图

Untitled

时纪

• E 4.1

- 有傻子系统调用完不返回啊.....如果没有 `jr ra` , 系统调用将无法正确返回结果给用户程序, 用户程序可能会无限期地等待系统调用的结果, 调用者无法恢复执行

- 同理，也需要注意EPC的设置，比如 `tf->cp0_epc += 4;`，让返回时返回到下一条指令。而如果是延迟槽指令，`epc`为当前`pc`,不需要再加4

- E 4.2

- 想半天去哪个代码文件找`reg`的每个编号到底对应哪个寄存器，突然想起是MIPS规定好的.....

表 A.5: MIPS 通用寄存器

寄存器编号	助记符	用途
0	zero	值总是为 0
1	at	(汇编暂存寄存器) 一般由汇编器作为临时寄存器使用。
2-3	v0-v1	用于存放表达式的值或函数的整形、指针类型返回值
4-7	a0-a3	用于函数传参。其值在函数调用的过程中不会被保存。若函数参数较多，多出来的参数会采用栈进行传递
8-15	t0-t7	用于存放表达式的值的临时寄存器; 其值在函数调用的过程中不会被保存。
16-23	s0-s7	保存寄存器; 这些寄存器中的值在经过函数调用后不会被改变。
24-25	t8-t9	用于存放表达式的值的临时寄存器; 其值在函数调用的过程中不会被保存。当调用位置无关函数 (position independent function) 时，25 号寄存器必须存放被调用函数的地址。
26-27	k0-k1	仅被操作系统使用。
28	gp	全局指针和内容指针。
29	sp	栈指针。
30	fp 或 s8	保存寄存器 (同 s0-s7)。也可用作帧指针。
31	ra	函数返回地址。

Untitled

- E 4.10

- 修改权限位之后需要重新进行页面映射

- E 4.13

- 注意使用`alloc`和`map`相关函数时需要考虑虚拟地址页对齐的问题

上机准备



需时刻注意区分用户态和内核态

- 编写/使用函数的时候需要注意，有些函数只能在内核态使用！比如page_alloc，在用户态需要换为syscall_mem_alloc来实现
 - 使用系统调用相关函数时，注意调用 syscall_* 时的参数列表/类型和 sys_* 不一定相同
 - 分配空间、建立映射时，注意考虑用 page_* 还是 syscall_*

常用函数/用法

用函数/实现函数的第一步注意使用前提条件！

- *(vpt+i) & PTE_V 目录项有效性检查
- vpt & vpd 用法

(*vpd) [va>>22 (页目录的索引)] & (0xfff) 表示二级页表的物理地址，(*vpt)[va>>12] & (0xfff) 为 va 对应的物理页面地址，使用前记得提前判断有效位

- env_id2env 通过一个进程的 id 获取该进程控制块

```
1 | try(env_id2env(env_id, &env, 1));
```

- is_illegal_va 判断虚拟地址的有效性

```
1 | if(is_illegal_va(va)){
2 |     return -E_INVALID;
3 | }
```

- *((struct Trapframe *)KSTACKTOP - 1) 内核中寄存器快照的位置
- TAILQ_INSERT_TAIL 将进程插入调度队列，注意是TAIL

```
1 | if( env->env_status!=ENV_RUNNABLE && status==ENV_RUNNABLE ){
2 |     TAILQ_INSERT_TAIL(&env_sched_list, env, env_sched_link);
3 | }
```

- TAILQ_REMOVE 将进程移除调度队列

```
1 | TAILQ_REMOVE(&env_sched_list, curenv, env_sched_link);
```

- `page_lookup` 写时复制时检查原页面映射是否存在

```
1 | p = page_lookup( curenv->env_pgdir, srcva, NULL);  
2 | if( p == NULL ){  
3 |         return -E_INVAL;  
4 | }
```

- `perm = *(vpt+VPN(va))&0xffff;` 获取页目录项权限
- `perm |= PTE_D; perm &= ~PTE_COW;` 更改页目录项权限
- `user_panic` 用户态崩溃

往年教训

- 可能会涉及ipc的具体实现，可以考虑开结构体数组记录每次信息发送的相关值和一个记录是否完成的标记
 - 接收进程：首先查表，有无自己可以接受的信息，有的话就接收，设置发送进程状态为RUNNABLE 并正常退出，否则阻塞。
 - 发送进程：检查接收进程的状态，若阻塞，直接进程信息发送同时设置接收进程状态为RUNNABLE。若接收进程没有阻塞，将待发送的信息添加到信息表中，阻塞。

上机血泪教训

extra-1

- “仿照xxx部分进行实现”，有一些部分**不确定要不要保留就先保留！！**（很有可能涉及到一些隐性机制的实现）

exam-2

“由俭入奢易，由奢入俭难”，某种程度上也适用于代码编写。用最**简单直接精炼**的代码实现，不要为求安心无脑加一些**似是似非**的代码，容易导致一些意想不到的错误而且难找原因。一定要**百分百确定有漏洞才恰当地打补丁**。



◀ BUAA-OS-Lab5 文件系统

BUAA-OS-Lab3 进程与异常 ▶

0 条评论

未登录用户 ▾



说点什么

① 支持 Markdown 语法

使用 GitHub 登录

预览

来做第一个留言的人吧!

Copyright © 2025 Roisy's Blog. Powered by Hexo. Theme by tufu9441.